Data Structures and Algorithms

Bonus Lab - Practice Tasks

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).

2. DEADLINE – One Week after assigned this.

3. SUBMISSION – This assignment needs to be submitted in a soft copy.

4. WHERE TO SUBMIT – Please visit your LMS.

5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000_assignment.zip and submit the ZIP file on the LMS.

Task 1: Reverse a Singly Linked List

- Reversing the links between nodes
- Using prev, current, and next pointers
- Modifying an existing linked list
- O(n) time complexity
- O(1) space complexity

Task 2: Merge Two Sorted Linked Lists

- Working with two linked lists
- Comparing nodes
- Merging two linked lists
- Maintaining sorted order
- O(n + m) time complexity
- O(1) auxiliary space

Task 3: Find the Middle Element of a Linked List

- Slow and fast pointer technique

- Traversing without knowing the size of the linked list
- Pointer-based problem solving
- O(n) time complexity
- O(1) space complexity

Task 4: Detect a Cycle in a Linked List

- Two-pointer technique
- Floyd's Cycle Detection Algorithm
- Using slow and fast pointers
- Detecting whether a linked list contains a cycle
- O(n) time complexity
- O(1) space complexity

Task 5: Find the Nth Node from the End

Given a linked list and an integer n, find the nth node from the end of the linked list without calculating the length of the linked list.

- Two-pointer technique
- Maintaining a fixed distance between two pointers
- Efficient linked list traversal
- O(n) time complexity
- O(1) space complexity

Task 6: Check if a Linked List is a Palindrome

Determine whether the elements of a linked list form a palindrome.

Example:

1 → 2 → 3 → 2 → 1

Output:

Palindrome

- Finding the middle of a linked list
- Reversing part of a linked list
- Comparing two parts of a linked list
- Combining multiple linked-list techniques
- O(n) time complexity
- O(1) auxiliary space

Task 7: Reverse a Doubly Linked List

Reverse a doubly linked list by correctly updating both the next and prev pointers.

Example:

10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ 50

After reversal:

50 ⇄ 40 ⇄ 30 ⇄ 20 ⇄ 10

- Manipulating next and prev pointers
- Reversing a doubly linked list
- Updating head and tail
- O(n) time complexity
- O(1) space complexity

Task 8: Josephus Problem Using a Circular Linked List

There are n people standing in a circle. Starting from the first person, eliminate every kth person until only one person remains. Use a Circular Linked List to solve the problem.

Example:

n = 7
k = 3

Determine the position of the last remaining person.

Students learn:

- Practical application of circular linked lists
- Circular traversal
- Repeated node deletion
- Pointer manipulation
- Problem-solving using circular linked lists
